Python Cheatsheet

Essential syntax, structures, and patterns for modern Python development

This cheatsheet provides quick reference to fundamental Python syntax and patterns, including data structures, functions, and frequently used standard library modules. Examples use Python 3.10+ and prefer modern idioms (f-strings, pathlib, dataclasses, type hints) over legacy alternatives.

Core Syntax	Functions & Control	Data Structures
Operators, data types, variables	Definitions, parameters, control flow	Lists, dicts, sets, tuples
Advanced Patterns	**File & Package Management**	
OOP, decorators, comprehensions	I/O operations, virtual environments	

Basics: Operators & Data Types

Operator Precedence

```
**  *  /  //  %  +  -  <<  >>  &  ^  |  ==  !=  <  <=  >  >=  is  in  not  and  or
```

Parentheses override default precedence. Prefer explicit parentheses for clarity.

Core Data Types

```
# Numbers
x = 42          # int
y = 3.14        # float
z = 1_000_000   # underscores for readability

# Strings (immutable)
s = "hello"
t = f"{x + y:.2f}"  # f-string: "43.14"

# Boolean
b = True
n = None        # singleton for "no value"

# Collections
lst = [1, 2, 3]
tup = (1, 2, 3)
d = {"key": "value"}
s = {1, 2, 3}
```

Functions: Definition & Lambda

Functions allow you to encapsulate reusable blocks of code.

```
# Define a function to print a greeting
def greet(name):
    print(f"Hello, {name}")

# call the function with an argument
greet("Alice")

# Functions can return values
def add(a, b):
    return a + b
result = add(5, 3)

# Lambda functions are small, anonymous functions for simple expressions.
# Useful for sorting, filtering, and as arguments to other functions.
add = lambda x, y: x + y
print(add(5, 3))  # 8

# Type hints (optional) improve readability and enable static analysis.
def greet(name: str) -> None:
    print(f"Hello, {name}")

# Docstrings document what a function does.
def add(a, b):
    """Return the sum of a and b."""
    return a + b
```

Lists & Tuples

Lists (Mutable)

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list[0])  # 1
print(my_list[-1])
print(len(my_list))
print(my_list[1:3])  # [2, 3]
```

Tuples (Immutable)

```
# Tuples are used to return multiple values, to hold heterogeneous
# data, and as dict keys (because they are immutable).
point = (10, 20)
print(point[0])
print(len(point))

# Tuple unpacking
x, y = point
print(f"x={x}, y={y}")

# Swap two variables using tuple unpacking
a, b = 1, 2
a, b = b, a
print(f"a={a}, b={b}")
```

Dictionaries: Key-Value Pairs

Dictionaries store data in unordered key-value pairs, allowing efficient lookup by key.

```
my_dict = {
    'name': 'Alice',
    'age': 25,
    'city': 'NYC'
}
print(my_dict['name'])
my_dict['age'] = 26
my_dict['email'] = 'alice@example.com'
print(len(my_dict))
print(my_dict.keys())
print(my_dict.values())
print(my_dict.items())
for key, value in my_dict.items():
    print(f"{key}: {value}")
print(my_dict.get('name', 'default'))
print(my_dict.get('phone', 'default'))
```

Sets: Unique Collections

Sets are unordered collections of unique items, useful for membership testing and eliminating duplicates.

```
# Create sets
s = {1, 2, 3}
s = set([1, 2, 3])
print(len(s))
print(1 in s)
print(4 in s)
s.add(4)
s.add(2)
print(s)
s.remove(3)
print(s)
# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b)
print(a & b)
print(a - b)
print(a ^ b)
```

Control Flow

Control flow statements determine the order in which code instructions are executed.

1 — Conditionals

```
# if/elif/else
x = 10
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")

# Ternary operator
status = "positive" if x > 0 else "non-positive"
print(status)

# Truthiness: these are falsy
# False, 0, 0.0, "", [], {}, set(), None
if not []:
    print("empty list is falsy")
```

2 — For Loops

```
# for loop
for i in range(5):
    print(i)

# range(start, stop, step)
for i in range(2, 10, 2):
    print(i)

# iterate over a list
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# enumerate gives index and value
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")

# break and continue
for i in range(10):
    if i == 3:
        continue
    if i == 7:
        break
    print(i)
```

3 — While Loop

```
# while loop
count = 0
while count < 5:
    print(count)
    count += 1

# Infinite loop with break
while True:
    response = input("Enter q to quit: ")
    if response == "q":
        break
    print("You entered:", response)
```

Comprehensions: Elegant Iteration

Comprehensions offer a concise way to create lists, sets, or dictionaries based on existing iterables.

```
# List comprehension: create a new list from an existing one.
# Syntax: [expr for item in iterable if condition]
squares = [x**2 for x in range(10)]
print(squares)
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Filter even numbers
evens = [x for x in range(20) if x % 2 == 0]
print(evens)
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Apply a function to each element
words = ["hello", "world", "python"]
upper = [w.upper() for w in words]
print(upper)
# ['HELLO', 'WORLD', 'PYTHON']

# Set comprehension: removes duplicates.
# Syntax: {expr for item in iterable if condition}
unique_lengths = {len(w) for w in words}
print(unique_lengths)
# {5, 6}

# Dictionary comprehension: create a new dictionary from an iterable.
# Syntax: {key_expr: value_expr for item in iterable if condition}
word_lengths = {w: len(w) for w in words}
print(word_lengths)
# {'hello': 5, 'world': 5, 'python': 6}

# Swap keys and values.
# Syntax: {value_expr: key_expr for item in iterable if condition}
swapped = {v: k for k, v in word_lengths.items()}
print(swapped)
# {5: 'world', 6: 'python'}

# Nested comprehension: flatten a matrix.
# Syntax: [expr for outer in iterable for inner in iterable]
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [x for row in matrix for x in row]
print(flat)
# [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Generator expression: lazy evaluation, saves memory.
# Syntax: (expr for item in iterable if condition)
gen = (x**2 for x in range(1000000))
print(next(gen))
print(next(gen))
# 0
# 1
```

String Formatting: Modern Approach

F-Strings (Python 3.6+)

```
name = "Alice"
age = 25
pi = 3.14159
print(f"Hello, {name}")
print(f"{age}")
print(f"{pi:.2f}")
print(f"{name.upper()}")
print(f"{age + 1}")
print(f"{'yes' if age > 18 else 'no'}")
print(f"{pi:10.2f}")
print(f"{pi:<10.2f}")
print(f"{pi:^10.2f}")
print(f"{pi:e}")
print(f"{1000000:,}")
print(f"{0.25:%}")
print(f"{255:b}")
print(f"{255:x}")
print(f"{255:o}")
```

Format Numbers

```
# Debugging (Python 3.8+)
x = 42
name = "Alice"
print(f"{x=}")
# x=42
print(f"{name=}")
# name='Alice'
print(f"{x=:>10}")
# x=        42
print(f"{x=:<10}")
# x=42
print(f"{x=:^10}")
# x=   42
print(f"{x=:=10}")
# x=        42
print(f"{x=:=+10}")
# x=       +42
```

String Manipulation

Python offers various methods to work with and modify strings.

```
# Strings are immutable: every operation returns a new string.
s = "  hello, world!  "
print(len(s))
print(s.strip())
print(s.lstrip())
print(s.rstrip())
print(s.upper())
print(s.lower())
print(s.replace("l", "L"))
print(s.strip().capitalize())
print(s.strip().split(", "))
print("-".join(["a", "b", "c"]))
print("hello".startswith("he"))
print("hello".endswith("lo"))
print("hello".find("ll"))
print("hello".count("l"))
print("hello".replace("l", "L", 1))
print("  ".join("abc"))
```

Regular Expressions

Regular expressions (regex) are powerful tools for pattern matching and text parsing.

```
import re

# Compile a pattern for better performance if used multiple times.
pattern = re.compile(r'\d+')
print(pattern.findall("abc123def456"))
# ['123', '456']

# Search for the pattern in a string.
text = "The quick brown fox jumps over the lazy dog"
print(pattern.search("abc123def456"))
# <re.Match object; span=(3, 6), match='123'>

# Find all matches.
print(pattern.findall("abc123def456"))
# ['123', '456']

# Replace all matches.
print(pattern.sub("#", "abc123def456"))
# abc#def#

# Split by the pattern.
print(pattern.split("abc123def456"))
# ['abc', 'def', '']

# Groups: use parentheses to capture specific parts of the match.
pattern = re.compile(r'(\d+)([a-z]+)')
print(pattern.search("abc123def456"))
# <re.Match object; span=(3, 6), match='123def'>

# Find all. Returns a list of all non-overlapping matches.
# Returns a list of tuples for grouped patterns.
print(pattern.findall("abc123def456"))
# [('123', 'def'), ('456', 'ghi')]

# Common patterns.
# \d+      one or more digits
# \w+      one or more word characters (letters, digits, underscore)
# \s+      one or more whitespace characters
# [a-z]+   one or more lowercase letters
# [^a-z]   any character NOT in the range
# .*       any character (greedy)
# .*?      any character (lazy)
# ^        start of string
# $        end of string
# \b       word boundary
# \B       not a word boundary
# (?:...)  non-capturing group
# (?=...)  lookahead
# (?!...)  negative lookahead
# (?<=...) lookbehind
# (?<!...) negative lookbehind
# |        alternation (or)
# \1       backreference to group 1
# \D       non-digit
# \W       non-word character
# \S       non-whitespace
# \A       start of string (like ^)
# \Z       end of string (like $)
# \n       newline
# \t       tab
# \r       carriage return
# \\       literal backslash
# \.       literal dot
# \*       literal asterisk
# \+       literal plus
# \?       literal question mark
# \[       literal [
# \]       literal ]
# \(       literal (
# \)       literal )
# \{       literal {
# \}       literal }
# \|       literal |
# \^       literal ^
# \$       literal $
# \/       literal /
# \:       literal :
# \;       literal ;
# \=       literal =
# \!       literal !
# \<       literal <
# \>       literal >
# \,       literal ,
# \-       literal -
# \#       literal #
# \%       literal %
# \&       literal &
# \@       literal @
# \`       literal `
# \~       literal ~
# \"       literal "
# \'       literal '
# \_       literal _
# \+       literal +
# \*       literal *
# \?       literal ?
```